

ASFA – A Simple Feed Aggregator

4th-Year Design Project

Project Overview

The goal of this project is to develop a lightweight, extensible social media feed aggregator (ASFA) that selectively ingests and filters content from multiple social media (SM) providers and exposes the curated content via an embeddable HTML `<iframe>` for use on websites.

The system will ultimately replace the current third-party solution backing msrg.org (currently powered by the free tier of elfsight.com) and, eventually, also support qsc.ca. The solution should be developed as an open-source project under a permissive license and hosted via the MSRG GitHub repository. The end goal is to run the aggregator within a Docker container, making it easy to deploy and scale. A test harness should be provided; future design projects may build and extend ASFA.

At present, we are unaware of any open-source feed aggregator that fully meets our requirements. The project is considered successful when ASFA can serve feed content from **LinkedIn**, **X (formerly Twitter)**, and **Bluesky**, and is actively used in production by the MSRG website. Further requirements list available upon project signup.

Project Phases and Requirements

1. Preliminary Analysis

- Analyze the capabilities of existing commercial feed aggregators such as Elfsight, and at least three other leading platforms.
- Identify and evaluate open-source alternatives, if any, and compare their features to those of commercial services and our specific requirements.

2. System Architecture & Core Functionality

- Design and implement a modular, extensible architecture that allows selective ingestion of feeds from LinkedIn (initially), X, and Bluesky.
- Provide a standardized HTML `<iframe>` output for easy integration into any website.
- Ensure extensibility to support additional social media platforms in the future.

3. Content Filtering & Curation

- Implement a basic *Content Curation Model (CCM)*:
 - For example, in the case of LinkedIn, an organizational account is created where designated administrators curate content (e.g., repost selected posts or create original content).
 - The aggregator only serves curated posts via the iframe.
- Eliminate duplicate posts on the same as well as across SMs and only show one.
- Support the selective removal of content that initially passed curation but is later deemed inappropriate or irrelevant.
- Provide a mechanism to manually add content not originating from any SM feed—such as special announcements—to be displayed via the iframe.
- Optional: Implement a Filtering Capability based on LinkedIn tags et al. that selectively ingests feed content and makes it available via the iframe

4. Multi-Site Support & Scalability

- Design the system to support multiple independent sites, each with its own curated feed.
- Ensure the system can scale as the number of backed sites grows, potentially via lightweight replication or instance deployment strategies.

5. Front-end

- Support via embedding an HTML iframe in the target website (cf. msrg.org as an example; eventually, also qsc.ca)
- Seamless support for reddening on desktop, tables, and mobile (i.e., the layout in the iframe needs to support these modalities and prominent browsers (i.e., Chrome, Firefox, and Safari))
- Aesthetic presentation
 - Design for aesthetically and effectively mixing SM provider feed content
 - Design for aesthetically, organized and well-structured SM provider feed rendering in the iframe and support capabilities to scroll through the various feeds on the target site (cf. the widget on msrg.org that allows for horizontal scrolling through the feed history)

6. Technical Implementation

- The system should be implemented in **Python** or **Node.js** and run within a **Docker** container.
- All code must be **open-source** and documented for community use and contributions.
- Support for configurable refresh frequencies (e.g., check for post updates every 1h or even only once every day)
- Support a management backend (that includes the OAuth routine). The emphasis should be on ease of maintainability once ASAF is deployed.

6. Getting started with LinkedIn

- How to get LinkedIn API access (this will have to be enabled for MSRG maintainers):
<https://learn.microsoft.com/en-us/linkedin/shared/authentication/getting-access?context=linkedin%2Fcontext>
- And here is a sample application:
<https://learn.microsoft.com/en-us/linkedin/shared/authentication/sample-applications?source=recommendations>

Deliverables

- A running prototype that supports feed ingestion from LinkedIn, with a functional iframe output.
- Documentation of the system architecture and deployment.
- Comparative analysis of existing feed aggregators and open-source tools.
- Publicly available codebase on GitHub under the MSRG account.